

autolisp-script – Batch REPL and Remote I/O Specification

Pascal Bourguignon

April 27, 2026

Contents

1	Objectif	2
2	Principes	3
3	Répertoire de session	3
4	Modèle d'exécution	4
5	Rôles	4
5.1	Shell	4
5.2	Runtime AutoLISP	5
6	Fonctions d'I/O shadowées	5
7	Modèle de lecture	6
7.1	Primitive de base	6
7.2	Construction de read	6
8	Modèle de sortie	7
9	États publiés dans status.txt	7
10	Canal stdin.txt	8
11	Canal control.txt	9
12	Heartbeat	9

13 Boucle principale	10
14 Arrêt propre	10
15 Interruptions	11
15.1 Principe	11
15.2 Interruption coopérative	11
15.3 Limites	11
16 Invariants de sûreté	12
17 Cas d'usage visés	12
17.1 REPL distant	12
17.2 read interactif	12
17.3 Mini-REPL Lisp côté utilisateur	13
18 Questions ouvertes	13

1 Objectif

Ce document spécifie le protocole de communication entre :

- le client shell = `autolisp-script/autolisp`
- le runtime AutoLISP embarqué dans `run-common.lsp`

Le protocole vise d'abord BricsCAD macOS en mode batch, mais il est formulé de manière assez générale pour être réutilisé ailleurs.

L'idée centrale est la suivante :

- il n'est pas utile de distinguer fondamentalement les "requêtes normales" et les "entrées utilisateur"
- tout passe par un même modèle d'entrées / sorties Lisp distantes
- les primitives d'I/O AutoLISP sont shadowées pour dialoguer avec le shell à travers des fichiers

Autrement dit, le shell n'envoie pas seulement des "commandes" à un REPL ; il devient le terminal distant du runtime Lisp.

2 Principes

- Le shell et AutoLISP communiquent uniquement par fichiers.
- Le protocole est synchrone et piloté par un état explicite.
- Les écritures shell vers les canaux d'entrée et de contrôle doivent être atomiques.
- Les écritures AutoLISP vers les canaux de sortie doivent être ordonnées et déterministes.
- Le socle de lecture doit être la ligne, pas directement la forme Lisp complète.
- `read` est reconstruit au-dessus de `read-line` et peut consommer plusieurs lignes si la forme est incomplète.

3 Répertoire de session

Chaque session possède un répertoire de travail dédié, noté `WORKDIR`.

Fichiers réservés :

- `status.txt` : état courant du runtime
- `stdin.txt` : entrée distante fournie par le shell
- `stdout.txt` : sortie distante normale
- `stderr.txt` : sortie d'erreur distante
- `control.txt` : commandes hors bande
- `heartbeat.txt` : battement de vie optionnel
- `render.txt` : rendu intermédiaire d'objets Lisp
- `run-common.lsp` : runtime AutoLISP généré
- `run.scr` : script de démarrage BricsCAD

Remarque :

- les noms exacts peuvent encore évoluer
- le point important est la séparation logique entre état, entrée, sortie et contrôle

4 Modèle d'exécution

Le shell lance BricsCAD avec un script minimal du type :

```
._COMMANDLINE  
(load ".../run-common.lsp")  
._QUIT _Y
```

Le rôle de `run.scr` est volontairement minimal :

- préparer la ligne de commande
- charger `run-common.lsp`
- quitter BricsCAD quand `run-common.lsp` rend la main

La vraie logique de protocole vit dans :

- le shell
- `run-common.lsp`

5 Rôles

5.1 Shell

Le shell :

- crée `WORKDIR`
- initialise les fichiers de session
- lance BricsCAD
- écrit l'entrée distante dans `stdin.txt`
- écrit les commandes hors bande dans `control.txt`
- lit `stdout.txt` et `stderr.txt`
- attend les transitions de `status.txt`
- tue le process BricsCAD en dernier recours

5.2 Runtime AutoLISP

Le runtime AutoLISP :

- shadowe certaines fonctions d'I/O
- publie son état dans `status.txt`
- lit les lignes entrantes depuis `stdin.txt`
- écrit les sorties dans `stdout.txt` et `stderr.txt`
- surveille `control.txt`
- maintient éventuellement un `heartbeat.txt`
- termine proprement la session lorsque l'arrêt est demandé

6 Fonctions d'I/O shadowées

Le protocole repose sur le shadowing des fonctions interactives ou semi-interactives.

Fonctions cibles prioritaires :

- `read-line`
- `read`
- `princ`
- `print`
- `prin1`
- `prompt`

Fonctions à considérer ensuite :

- `read-char`
- `getstring`
- `getint`
- `getreal`

- `getpoint` ou autres primitives CAD interactives, si elles doivent être prises en charge

Contraintes :

- le shadowing ne doit concerner que les flux interactifs distants
- les appels explicites sur un vrai file handle doivent continuer à fonctionner comme attendu

Exemple :

- `(read f)` doit continuer à lire dans `f`
- `(read)` sans argument peut être redirigé vers l'entrée distante

7 Modèle de lecture

7.1 Primitive de base

La primitive de base du protocole est la lecture de ligne distante.

Le runtime appelle une fonction interne du type :

- `autolisp-remote-read-line`

Cette fonction :

- attend qu'une ligne soit disponible dans `stdin.txt`
- la consomme
- la renvoie comme chaîne

7.2 Construction de `read`

`read` ne doit pas être pensé comme un aller-retour unique.

`read` doit :

- accumuler une ou plusieurs lignes obtenues via `autolisp-remote-read-line`
- poursuivre tant que la forme Lisp n'est pas syntaxiquement complète
- retourner l'objet Lisp lu quand l'équilibre est atteint

Cela permet de supporter naturellement des cas comme :

```
(loop (print (eval (read))))
```

et aussi la saisie multi-ligne de listes ou de formes imbriquées.

8 Modèle de sortie

Toutes les sorties distantes passent par des helpers du type :

- `autolisp-remote-write-stdout`
- `autolisp-remote-write-stderr`

Les fonctions shadowées comme `princ`, `print`, `prin1` et `prompt` utilisent ensuite ces helpers.

Le protocole peut distinguer :

- la sortie standard
- la sortie d'erreur
- éventuellement le "result" de REPL

Mais cette distinction est une convention de présentation côté shell, pas une nécessité fondamentale du protocole.

9 États publiés dans `status.txt`

Les états suivants sont réservés :

- `BOOTING`
- `READY`
- `WAITING-INPUT`
- `RUNNING`
- `STOPPING`
- `STOPPED`
- `FAILED`

Sémantique :

- `BOOTING` : le runtime démarre
- `READY` : le runtime est initialisé et disponible

- **WAITING-INPUT** : le runtime attend une ligne ou une suite de lignes venant du shell
- **RUNNING** : le runtime traite actuellement une forme ou une commande
- **STOPPING** : un arrêt a été demandé et accepté
- **STOPPED** : la boucle runtime est terminée
- **FAILED** : le runtime a rencontré une erreur fatale de protocole

Le shell ne doit pas déduire l'état du système à partir des seules sorties texte.

10 Canal `stdin.txt`

`stdin.txt` transporte les lignes d'entrée fournies par le shell.

Règles :

- le shell écrit une ou plusieurs lignes dans un fichier temporaire
- le shell publie ces lignes atomiquement vers `stdin.txt`
- le runtime consomme les lignes publiées

Le protocole de consommation peut être :

- soit destructif, en supprimant le fichier après lecture
- soit séquentiel, avec un curseur logique

Le choix exact est une question d'implémentation ; la spec impose surtout :

- pas de lecture partielle d'un fichier en cours d'écriture
- pas d'ambiguïté sur la disponibilité des lignes

11 Canal `control.txt`

`control.txt` est réservé au plan hors bande.

Commandes prévues :

- PING
- SHUTDOWN
- INTERRUPT

Remarques :

- SHUTDOWN demande un arrêt propre de la boucle runtime
- INTERRUPT ne peut être qu'une interruption coopérative
- PING sert au diagnostic ou à la supervision

Le runtime doit consommer ces commandes indépendamment du flux normal de `stdin.txt`.

12 Heartbeat

`heartbeat.txt` est optionnel mais recommandé.

Le runtime peut y écrire périodiquement :

- un compteur
- ou un timestamp logique

Le shell peut s'en servir pour distinguer :

- un runtime vivant mais inactif
- un runtime figé

Le heartbeat complète `status.txt` mais ne le remplace pas.

13 Boucle principale

Le runtime peut être décrit conceptuellement ainsi :

1. publier `BOOTING`
2. initialiser les helpers d'I/O distants
3. publier `READY`
4. entrer dans une boucle
5. publier `WAITING-INPUT`
6. attendre soit de l'entrée dans `stdin.txt`, soit une commande dans `control.txt`
7. si une entrée arrive, publier `RUNNING`, la lire, l'évaluer, produire les sorties, puis revenir à `READY`
8. si `SHUTDOWN` arrive, publier `STOPPING`, sortir de la boucle, puis publier `STOPPED`

14 Arrêt propre

Le mode normal d'arrêt est :

- soit une commande explicite de l'utilisateur, par exemple (`quit`)
- soit une commande hors bande `SHUTDOWN`

Le scénario attendu est :

1. le runtime détecte la demande d'arrêt
2. il termine proprement ses écritures
3. il publie `STOPPING`
4. il sort de la boucle principale
5. il publie `STOPPED`
6. `run-common.lsp` rend la main

7. `run.scr` reprend et exécute `._QUIT _Y`

Cette propriété est importante :

- `run.scr` ne doit pas piloter la boucle REPL
- il ne fait qu'enclencher le chargement puis fermer BricsCAD après retour de `run-common.lsp`

15 Interruptions

15.1 Principe

AutoLISP ne fournit pas ici de mécanisme de préemption asynchrone général utilisable.

Donc, une interruption ne peut pas être forte au sens système ; elle ne peut être que coopérative.

15.2 Interruption coopérative

Le runtime peut exposer une fonction du type :

- `autolisp-check-control`

Cette fonction :

- lit `control.txt`
- détecte `INTERRUPT`
- positionne un drapeau global, par exemple `*AUTOLISP_INTERRUPT*`

Les boucles coopératives peuvent alors consulter ce drapeau et sortir proprement.

15.3 Limites

Les cas suivants ne sont pas interrompables proprement par le protocole seul :

- forme Lisp monolithique longue
- boucle utilisateur non coopérative

- commande CAD bloquante

Dans ces cas, le shell doit pouvoir :

- constater le timeout
- conserver `WORKDIR`
- tuer le process BricsCAD

16 Invariants de sûreté

- Le shell n'écrit jamais `stdin.txt` ou `control.txt` sans publication atomique.
- Le runtime ne traite jamais de données partiellement publiées.
- Le shell n'écrit pas de nouvelle entrée si le runtime n'est pas dans un état compatible.
- Les sorties sont stables quand l'état repasse de `RUNNING` à `READY` ou `STOPPED`.
- Les commandes hors bande restent séparées du flux normal d'entrée.

17 Cas d'usage visés

17.1 REPL distant

Le shell lit une ligne utilisateur, la publie dans `stdin.txt`, puis affiche les sorties distantes.

17.2 read interactif

Si le code exécuté appelle `(read)`, le runtime lit autant de lignes distantes que nécessaire jusqu'à obtenir une forme complète.

Par exemple, si l'utilisateur tape :

42

alors `(read)` retourne la valeur Lisp 42.

Si l'utilisateur tape une liste incomplète, `read` continue à demander des lignes jusqu'à fermeture équilibrée.

17.3 Mini-REPL Lisp côté utilisateur

Le protocole doit supporter naturellement une boucle comme :

```
(loop (print (eval (read))))
```

sans protocole spécifique "requête métier".

18 Questions ouvertes

- faut-il introduire une granularité caractère via `read-char`, ou rester strictement ligne par ligne dans un premier temps
- format exact de consommation de `stdin.txt`
- stratégie précise de purge de `control.txt`
- niveau de compatibilité attendu avec les fonctions interactives spécifiques CAD
- manière la plus propre de distinguer `stdout`, `stderr` et "résultat REPL" côté UX shell